

BUSINESS ANALYTICS

OFFLINE PRACTICAL EXERCISES

PRESENTER:DRISHA.J

TOPIC 1: Write five user stories for an online food ordering app with clear acceptance criteria

User Story 1 - User Registration and Login

Title	User Story	Description	Acceptance Criteria (Given / When / Then)	Priori ty	Estima tion	Status	Dependcienes
1. User Registration and Login	As a new user, I want to create an account and log in so that I can securely access the food ordering app.	This feature allows users to register using mobile number or email and log in securely	Given the user is on the registration page, When they enter valid details and submit, Then an account should be created successfully. Given valid credentials, When the user logs in, Then they should be redirected to the home screen.	High	3 Story points	Completed	Database connection, API endpoint for authentication

User Story 2 - Browse and Search Restaurants

Title	User Story	Description	Acceptance Criteria (Given / When / Then)	Priority	Estimation	Status	Dependencies
2. Browse and Search Restaurants	As a user, I want to search and view restaurants near my location so that I can easily find where to order food.	This feature displays restaurants based on user's selected or current location.	Given the user has entered a location, When they search, Then a list of nearby restaurants should be displayed. Given the user changes the location, When search is updated, Then restaurant results should refresh automatically.	High	5 Story points	In progress	Location service API

User Story 3 - Add to Cart and Customize Order

Title	User Story	Description	Acceptance Criteria (Given / When / Then)	Priority	Esti matio n	Status	Dependcienes
3. Add to Cart and Customize Order	As a user, I want to add food items to my cart and customize them so that I can order food according to my preferences..	This feature lets users choose items, modify quantity, and add extra toppings or notes before checkout	Given the user views a menu, When they add an item, Then it should appear in the cart. Given an item is added, When the user updates quantity or adds toppings, Then the total amount should change accordingly.	Medium	5 Story points	Pending	Menu service, cart management API

User Story 4 - Secure Payment Integration

Title	User Story	Description	Acceptance Criteria (Given / When / Then)	Priority	Esti matio n	Status	Dependcienes
4. Secure Payment Integration	As a user, I want multiple payment options so that I can complete my purchase using my preferred method.	Enables payments through UPI, card, or cash on delivery with proper validation and confirmation	Given an order is ready for checkout, When a payment method is selected, Then the app should process it securely. Given the payment is successful, When confirmation is received, Then the order should move to “Placed” status.	High	8 Story points	In progress	Payment gateway API, order confirmation module

User Story 5 - Live Order Tracking and Notifications

Title	User Story	Description	Acceptance Criteria (Given / When / Then)	Priority	Estimation	Status	Dependencies
5 . Live Order Tracking and Notifications	As a customer, I want to track my order in real time so that I can know the current delivery status.	Displays live tracking of the delivery partner's location with push notifications for every order stage.	Given the order is placed, When the restaurant updates status, Then the tracking screen should show the new status. Given delivery starts, When the partner is nearby, Then the user should get a “Delivery arriving soon” notification.	High	8 Story points	Planned	Delivery API, notification service

User Story 6 - Rating and Feedback System

Title	User Story	Description	Acceptance Criteria (Given / When / Then)	Priority	Estimation	Status	Dependcienes
6. Rating and Feedback System	As a user, I want to rate my food and delivery experience so that I can help improve service quality.	Allows users to give ratings and comments after the order is delivered.	Given the order is delivered, When the user opens past orders, Then they should see a Rate Now button. Given feedback is submitted, When it is saved, Then the average rating for that restaurant should be updated.	Medium	4 Story points	Pending	Review API, user profile module

User Story 7 - Order History and Reorder

Title	User Story	Description	Acceptance Criteria (Given / When / Then)	Priority	Estimation	Status	Dependencies
7. Order History and Reorder	As a returning user, I want to view my past orders and reorder items easily.	Displays previous orders and provides quick reorder functionality.	Given the user navigates to “My Orders”, When they tap on a previous order, Then it should display order details. Given a reorder option, When selected, Then the items should be added to the cart automatically.	Low	3 Story points	Planned	Order database, cart module

TOPIC 2 - Create a Use Case Diagram & Activity Diagram for the food ordering app

Actors:

- 1.**Customer** (registered/unregistered) – browses, selects items, pays, tracks orders.
- 2.**Restaurant** – manages menu, views and accepts/rejects orders, updates order status.
- 3.**Delivery Person** – views orders assigned, updates delivery status, views earnings.
- 4.**Admin** – manages users, restaurants, orders and views reports.

Primary use-cases (customer side):

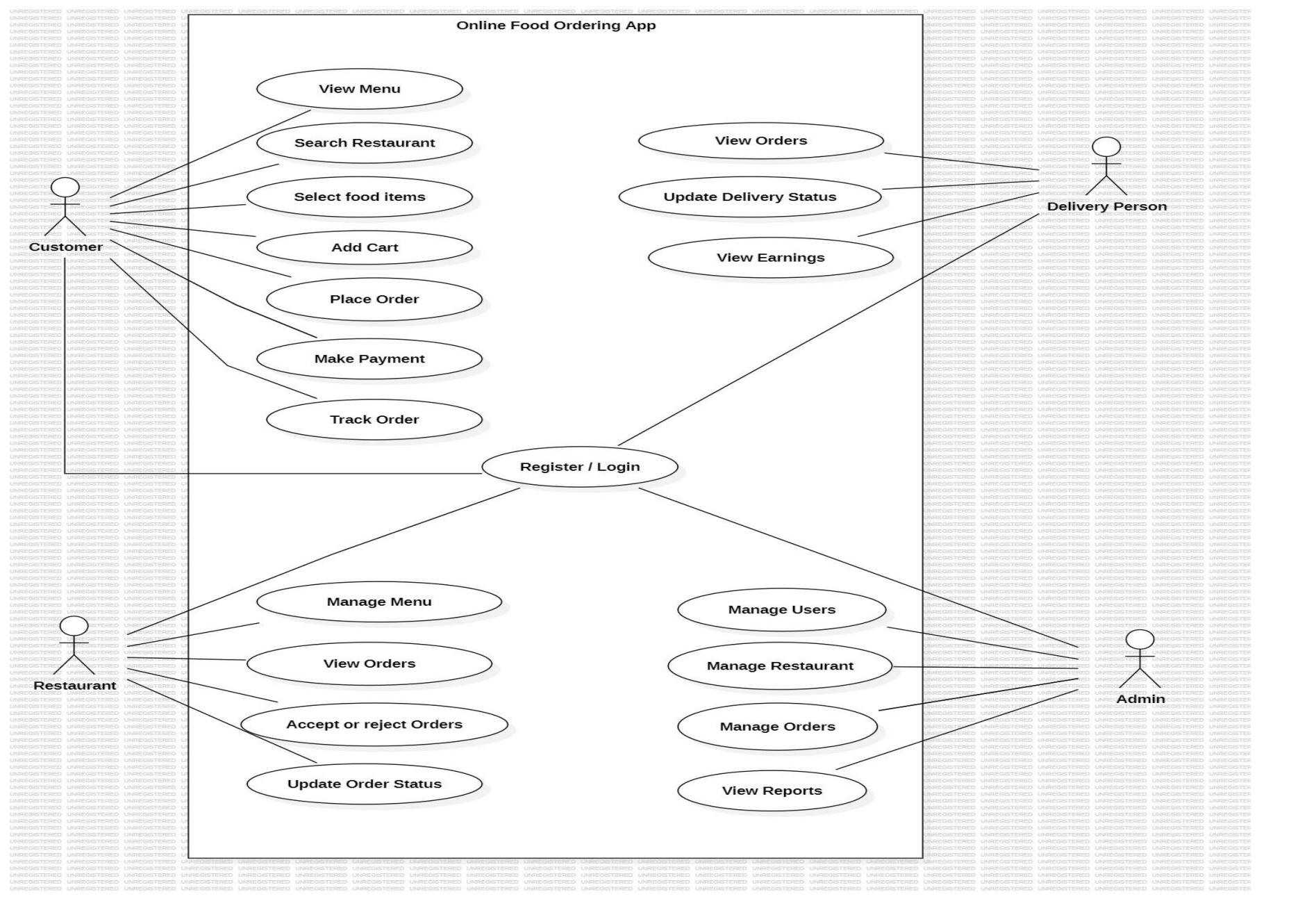
- View Menu -> Search Restaurant -> Select Food Items -> Add to Cart -> Place Order -> Make Payment -> Track Order.

Restaurant use-cases:

- Manage Menu -> View Orders -> Accept/Reject Orders -> Update Order Status.

Delivery & Admin use-cases:

- **Delivery Person:** View Orders -> Update Delivery Status -> View Earnings.
- **Admin:** Manage Users/Restaurants/Orders -> View Reports.



2. Activity Diagram — high-level customer order flow

Start- Register & Login

Decision: If login fails -> show “Invalid login or Password” -> user may retry.

Main path:

- Homepage -> Browse Restaurants -> View Menu & Select Item
- Add to Cart -> Place Order -> Make Payment

Decision after payment:

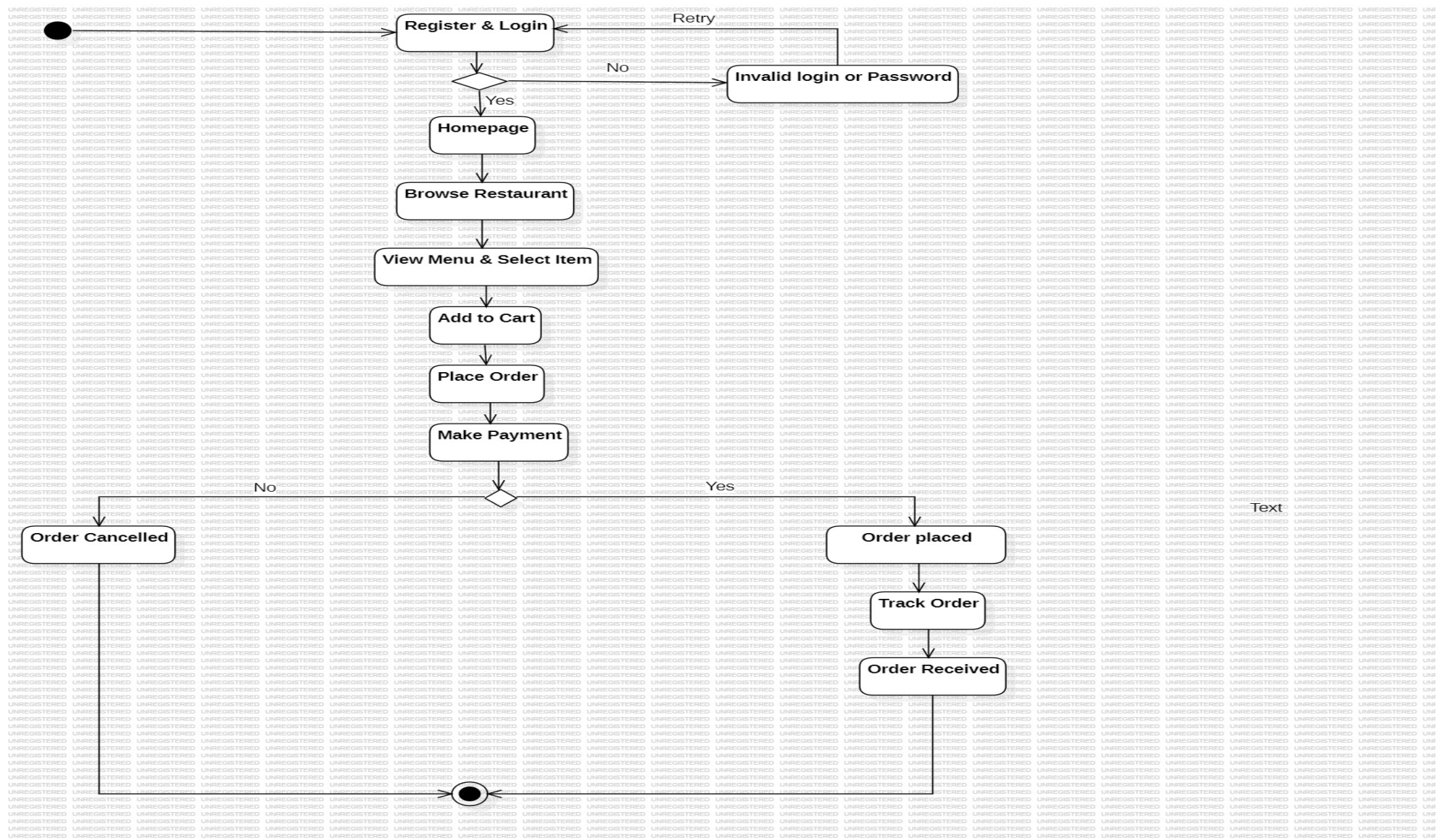
- **If payment succeeds:** Order placed -> Track Order -> Order Received and **end**.
- **If payment fails:** Order Cancelled and **end**.

Key alternate flows:

- Payment failure cancels order.
- **Implicit retries:** login retry and possibly retry payment.

Postconditions:

- **On success:** order recorded, restaurant notified, delivery process begins.
- **On failure:** order not recorded / marked cancelled, user notified.



3. Sequence Diagram — message-level interaction

Participants (left -> right): Customer -> Online Food Ordering System -> Restaurant Database -> Restaurant -> Payment Gateway -> Delivery Person

Step-by-step:

- **Customer selects a restaurant** (UI action -> system).
- **System fetches menu** from Restaurant Database.
- **Database returns menu** to system.
- **System displays menu** to customer.
- **Customer places order** and confirms the order.
- **System sends order** to the Restaurant.
- **Restaurant confirms order** back to System (accept or reject).
- **System sends confirmation** to Customer (indicating order accepted).
- **Customer makes payment** (initiates payment through system).
- **System confirms payment initiation** through Payment Gateway.
- **Payment Gateway verifies payment** (auth) and replies to System.
- **System sends payment confirmation** to Restaurant and Customer.
- **Restaurant receives order** officially and proceeds to prepare it
- **Delivery Person** later gets assigned/receives order.

sd Interaction1

Customer

Online Food Ordering system

Restaurant Dataase

Restaurant

Payment Gateway

Delivery Person

1 : selects restaurant

2 : fetches menu

3 : returns menu

4 : displays menu

5 : places order

6 : sends order

7 : confirms order

8 : sends confirmation

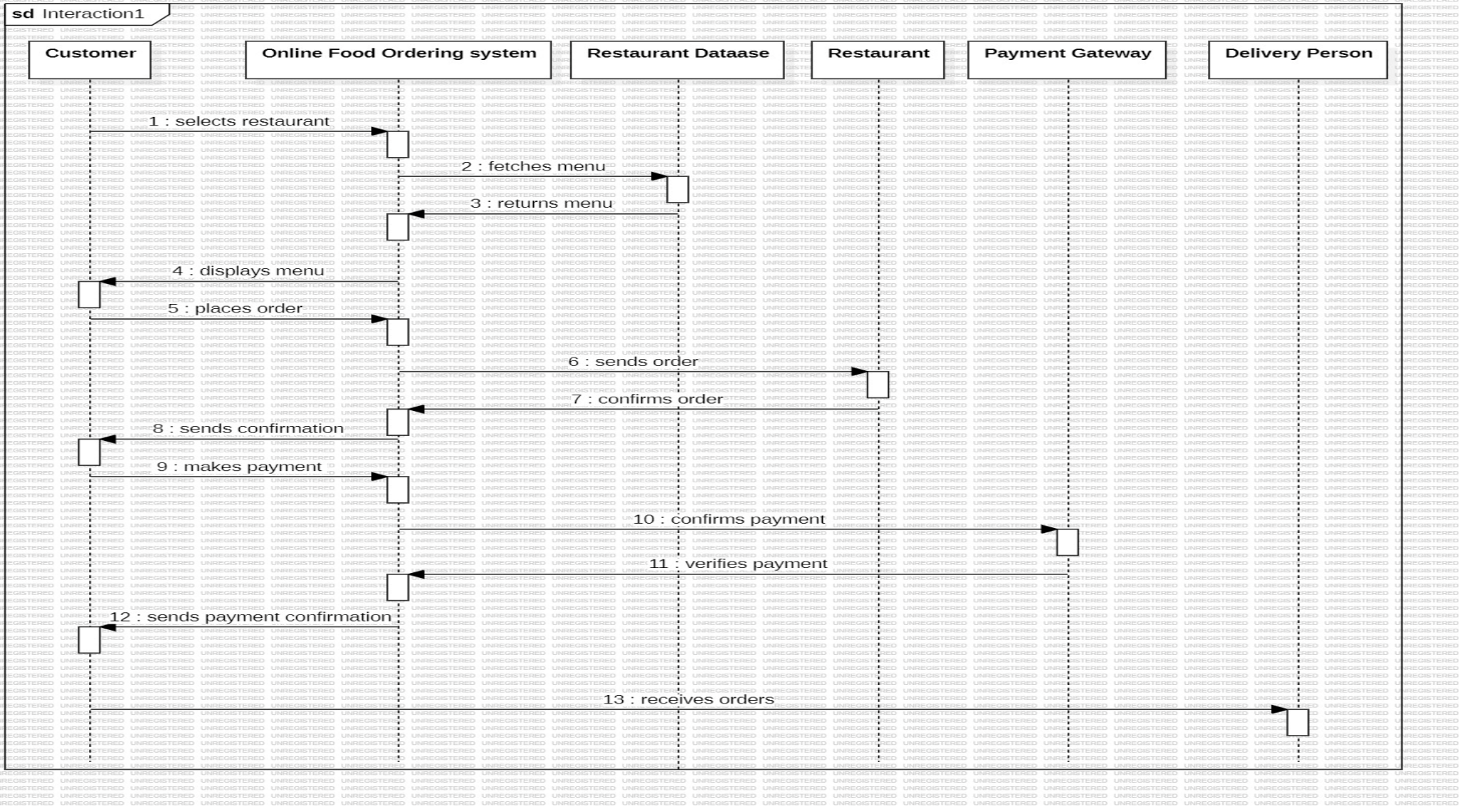
9 : makes payment

10 : confirms payment

11 : verifies payment

12 : sends payment confirmation

13 : receives orders



TOPIC 3 – Document an API Request – Response flow by analyzing a postman collection

1. Objective:

- The goal is to analyze and document how an API works from request initiation to response handling using a Postman collection.
- This ensures developers and testers can understand how data flows between client and server.

2. Tools & Inputs:

- **Postman** (to run and inspect API requests).
- A **Postman Collection** (contains predefined API endpoints)
- Authentication details (e.g: tokens, keys).

3. Typical API Flow (Request-Response):

- **Step 1: Client Sends Request**
- **Method:** e.g: POST
- **Endpoint URL:** e.g: <https://api.example.com/v1/order>
- **Headers(Key-Value):**
 - Content-Type: application/json
 - Authorization: Bearer <token>

Body (JSON):

```
{  
  "restaurantId": "1234",  
  "itemId": "5678",  
  "quantity": 2,  
  "paymentMethod": "UPI"  
}
```

Step 2: API Gateway / Server Receives Request

- Validates request format and authentication.
- Checks for required fields (e.g: restaurantId, itemId).
- Routes to the appropriate controller/service.

Step 3: Business Logic Execution

- Fetch menu & order details from database.
- Calculate order total, tax, delivery charges.
- Validate stock availability.
- Initiate payment processing .

Step 4: Server Sends Response

- **Status Code:** 201 Created (for success), 400 Bad Request, 401 Unauthorized etc.
- **Headers:**
 - Content-Type: application/json
- **Body** (Success example):

```
{  
  "orderId": "ORD7890",  
  "status": "Order Placed",  
  "amount": 450,  
  "message": "Your order has been placed successfully."  
}
```

Body (Error example):

```
{  
  "status": "Failed",  
  "errorCode": "PAYMENT_DECLINED",  
  "message": "Payment failed. Please try again."  
}
```


Step 5: Client Handles Response:

- If success -> display order confirmation, update UI.
- If failure -> show error message, allow retry.
- Log the response .

4. How to Document from Postman:

1. Import or Create Your Postman Collection

- Open Postman.
- If you already have a collection file (.json) click:
- File -> Import -> Upload Files -> Select your Postman collection file.(eg: Food Ordering App)

2.Open request tab - note **HTTP Method, API Endpoint, Headers, params, Auth,** and **Request Body**.

3.Click Send to view the response from the server.- capture status code, response body, and response time.

4.Click Documentation tab:

Add

- Request details
- Sample request & response
- Expected behavior
- Possible error responses
- Authentication type

5. Steps to Analyze an API in Postman

- **Open Postman** -> select the API from the collection.
- Check:
 - Method (GET, POST, PUT, DELETE)
 - URL
 - Headers
 - Body (JSON / form-data / params)
- Click **Send**.
- Check the **Response tab** for:
 - Status code
 - Time
 - JSON response
- Copy both request and response -> Document in the above format.

Final Output

- You now have:
- A clear explanation of what request was sent.
- A clear explanation of how the server responded.
- Both success and error flows documented.
- An easy table for future developers to understand quickly.

Step	Element	Description
1	Method	POST
2	URL	https://api.foodapp.com/v1/order
3	Headers	Content-Type: application/json; Authorization: Bearer Token
4	Request Body	JSON data with restaurantId, itemId, quantity, paymentMethod
5	Success Response Code	201 (Created)
6	Success Response Body	orderId, status, amount, message
7	Error Response Code	400 / 401
8	Error Response Body	Error code, message
9	Notes	Token must be valid, item must exist, payment must succeed.

THANK YOU

OFFLINE PRACTICAL EXERCISES

PRESENTED BY:DRISHA.J